CAMBRIDGE
International Education

# Cambridge International AS & A Level

**COMPUTER SCIENCE** **9618/22**

Paper 2 Fundamental Problem-solving and Programming Skills **May/June 2024**

MARK SCHEME

Maximum Mark: 75

**Published**

This mark scheme is published as an aid to teachers and candidates, to indicate the requirements of the examination. It shows the basis on which Examiners were instructed to award marks. It does not indicate the details of the discussions that took place at an Examiners' meeting before marking began, which would have considered the acceptability of alternative answers.

Mark schemes should be read in conjunction with the question paper and the Principal Examiner Report for Teachers.

Cambridge International will not enter into discussions about these mark schemes.

Cambridge International is publishing the mark schemes for the May/June 2024 series for most Cambridge IGCSE, Cambridge International A and AS Level and Cambridge Pre-U components, and some Cambridge O Level components.



© Cambridge University Press & Assessment 2024 

## Generic Marking Principles

These general marking principles must be applied by all examiners when marking candidate answers. They should be applied alongside the specific content of the mark scheme or generic level descriptions for a question. Each question paper and mark scheme will also comply with these marking principles.

GENERIC MARKING PRINCIPLE 1:

Marks must be awarded in line with:

- the specific content of the mark scheme or the generic level descriptors for the question
- the specific skills defined in the mark scheme or in the generic level descriptors for the question
- the standard of response required by a candidate as exemplified by the standardisation scripts.

GENERIC MARKING PRINCIPLE 2:

Marks awarded are always **whole marks** (not half marks, or other fractions).

GENERIC MARKING PRINCIPLE 3:

Marks must be awarded **positively**:

- marks are awarded for correct/valid answers, as defined in the mark scheme. However, credit is given for valid answers which go beyond the scope of the syllabus and mark scheme, referring to your Team Leader as appropriate
- marks are awarded when candidates clearly demonstrate what they know and can do
- marks are not deducted for errors
- marks are not deducted for omissions
- answers should only be judged on the quality of spelling, punctuation and grammar when these features are specifically assessed by the question as indicated by the mark scheme. The meaning, however, should be unambiguous.

GENERIC MARKING PRINCIPLE 4:

Rules must be applied consistently, e.g. in situations where candidates have not followed instructions or in the application of generic level descriptors.

GENERIC MARKING PRINCIPLE 5:

Marks should be awarded using the full range of marks defined in the mark scheme for the question (however; the use of the full mark range may be limited according to the quality of the candidate responses seen).

GENERIC MARKING PRINCIPLE 6:

Marks awarded are based solely on the requirements as defined in the mark scheme. Marks should not be awarded with grade thresholds or grade descriptors in mind.

© Cambridge University Press & Assessment 2024 

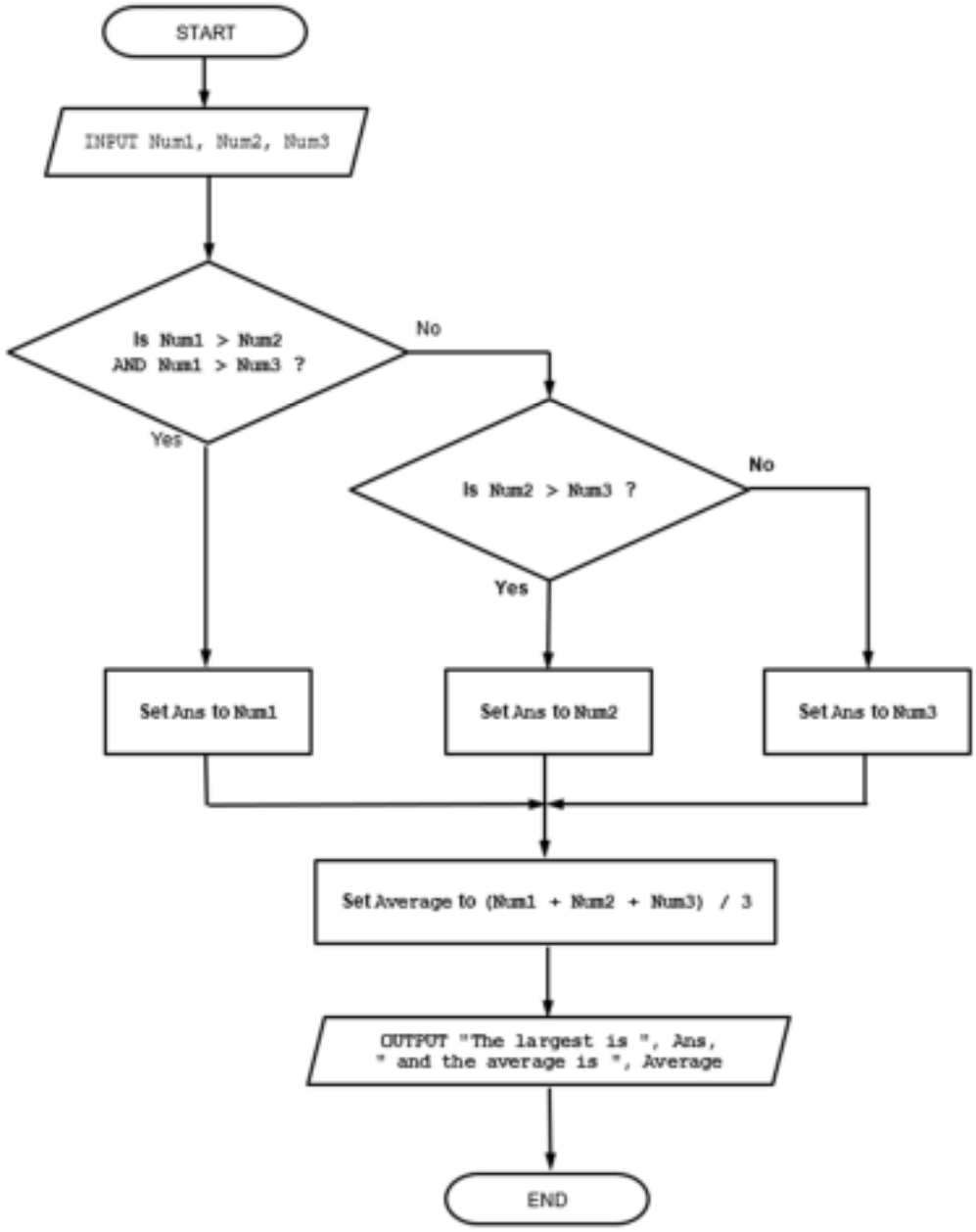
**Mark scheme abbreviations**

| | |
|---|---|
| / | separates alternative words / phrases within a marking point |
| // | separates alternative answers within a marking point |
| **underline** | actual word given must be used by candidate (grammatical variants accepted) |
| **max** | indicates the maximum number of marks that can be awarded |
| ( ) | the word / phrase in brackets is not required, but sets the context |

**Note:** No marks are awarded for using brand names of software packages or hardware.

© Cambridge University Press & Assessment 2024 

<table>
<tr><th>Question</th><th>Answer</th><th>Marks</th></tr>
<tr><td>1(a)</td><td>
<table>
<tr><th>Pseudocode example</th><th>Selection</th><th>Iteration</th><th>Input/Output</th></tr>
<tr><td>FOR Index ← 1 TO 10<br>  Data[Index] ← 0<br>NEXT Index</td><td></td><td>✓</td><td></td></tr>
<tr><td>WRITEFILE ThisFile, "****"</td><td></td><td></td><td>✓</td></tr>
<tr><td>UNTIL Level > 25</td><td></td><td>✓</td><td></td></tr>
<tr><td>IF Mark > 74 THEN<br>  READFILE OldFile, Data<br>ENDIF</td><td>✓</td><td></td><td>✓</td></tr>
</table>
One mark per row.
</td><td>4</td></tr>
<tr><td>1(b)</td><td>
<table>
<tr><th>Expression</th></tr>
<tr><td>MyInt ← INT (3.1415926)</td></tr>
<tr><td>MyChar ← MID ("Elwood", 3, 1)</td></tr>
<tr><td>Any of:<br>• MyString ← NUM_TO_STR (INT (27.509))<br>• MyString ← CHR (INT (27.509))<br>• MyString ← TO_UPPER( NUM_TO_STR( 27.509))<br>• MyString ← TO_LOWER( NUM_TO_STR( 27.509))</td></tr>
<tr><td>Any of:<br>• MyInt ← STR_TO_NUM ( RIGHT ("ABC123", 3))<br>• MyInt ← LENGTH ( RIGHT ("ABC123", 3))<br>• MyInt ← LENGTH ( LEFT ("ABC123", 3))</td></tr>
</table>
One mark per row
</td><td>4</td></tr>
<tr><td>1(c)</td><td>1 mark for stating a suitable way of documenting:<br>• <u>Identifier table</u><br>1 mark for giving one piece of information that should be recorded.<br>examples include:<br>Explanation of what (each) variable is used for<br>The purpose of (each) variable<br>An example of data values stored // Initialisation value</td><td>2</td></tr>
</table>

| Question | Answer | Marks |
|---|---|---|
| 2(a) |  <br>Mark points<br>1 Condition for selecting one of the input numbers as largest value<br>2 … and assign to `Ans`<br>3 Condition for selecting largest number for all three number input and assigning to `Ans`<br>4 Average calculated using `Num1`, `Num2` and `Num3` and stored in a variable. Reject use of DIV<br>5 Output of `Ans` and average in output symbol / parallelogram | **5** |

| Question | Answer | Marks |
|---|---|---|
| 2(b) | Example solutions:<br><br>`Flag ← GetStat()`<br>`WHILE Flag <> TRUE`<br>`  FOR Port ← 1 TO 3`<br>`  CALL Reset(Port)`<br>`  NEXT Port`<br>`  Flag ← GetStat()`<br>`ENDWHILE`<br><br>Alternative:<br><br>`REPEAT`<br>`  Flag ← GetStat()`<br>`  IF Flag <> TRUE THEN`<br>`  FOR Port ← 1 TO 3`<br>`  CALL Reset(Port)`<br>`  NEXT Port`<br>`  ENDIF`<br>`UNTIL Flag = TRUE`<br><br>One mark per point:<br>1 (Outer) conditional loop testing `Flag`<br>2 Correct assignment of `Flag` from `GetStat()` **in a loop**<br>3 (Inner) loop checking / counting port // Check if `Port` is different to 4<br>4 … loop for 3 iterations<br>5 a call to `Reset()` **in a loop** | **5** |

| Question | Answer | Marks |
|---|---|---|
| 3(a)(i) | Pseudocode:<br><br>`TYPE Component`<br>`  DECLARE Item_Num : INTEGER`<br>`  DECLARE Reject  : BOOLEAN`<br>`  DECLARE Stage  : CHAR`<br>`  DECLARE Limit_1  : REAL`<br>`  DECLARE Limit_2  : REAL`<br>`ENDTYPE`<br><br>Mark as follows:<br><br>1 One mark for `TYPE` and `ENDTYPE` statements<br>2 One mark for `Item_Num` **and** `Reject` fields<br>3 One mark for `Stage` field<br>4 One mark for Limit fields as `REAL` | **4** |

| Question | Answer | Marks |
|---|---|---|
| 3(a)(ii) | DECLARE Item : ARRAY [1:2000] OF Component//<br>DECLARE Item : ARRAY [2000] OF Component//<br>DECLARE Item : ARRAY [0:1999] OF Component<br><br>One mark per underlined phrase | **2** |
| 3(b) | One mark per point:<br><br>1 Allows for iteration / can use a loop **to access the records / data items**<br>2 Use of index to directly access a **record** in the array // Example of simplification of code e.g. use of dot notation Item[1].Stage<br>3 Simplifies the code / algorithm // Reduces duplication of code // **Program** easier to write / understand / maintain / test / debug // **Data items/record** easier to search / sort / manipulate | **3** |

| Question | Answer | Marks |
|---|---|---|
| 4 | Example solution:<br><br>```<br>PROCEDURE IsRA()<br>   DECLARE a, b, c : INTEGER<br><br>   OUTPUT "Input length of the first side"<br>   INPUT a<br>   OUTPUT "Input length of the second side"<br>   INPUT b<br>   OUTPUT "Input length of the third side"<br>   INPUT c<br><br>   IF (a * a = (b * b) + (c * c)) OR__<br>      (b * b = (a * a) + (c * c)) OR__<br>      (c * c = (a * a) + (b * b)) THEN<br>      OUTPUT "It is right-angled"<br>   ELSE<br>      OUTPUT "Not right-angled"<br>   ENDIF<br><br>ENDPROCEDURE<br>```<br><br>Mark as follows:<br><br>1. Procedure heading and ending **and** declaration of all variables used<br>2. Appropriate prompt and input for each length<br>3. **One** correct length test<br>4. **All three** length tests // selection of which test is required<br>5. Output one of two messages following a reasonable attempt at MP3 | **5** |

| Question | Answer | Marks |
|---|---|---|
| 5(a)(i) | One mark per error:<br><br>Syntax:<br>1. `NEXT Index` (should be `ENDWHILE)`<br>2. '&' used to concatenate an integer (in `OUTPUT` statement)<br><br>Other:<br>3. Accesses element outside range // Accesses element 0 | **3** |
| 5(a)(ii) | One mark per point:<br><br>Statement:<br>• The `OTHERWISE` statement<br><br>Explanation:<br>• The result of MOD 2 can only be 0 or 1 | **2** |
| 5(b) | Run-time | **1** |

| Question | Answer | Marks |
|---|---|---|
| 6(a) | Example solution:<br><br>`Function Trim(Name : STRING) RETURNS STRING`<br>`  CONSTANT Dots = "..."`<br>`  CONSTANT Space = " "`<br><br>`  IF LENGTH(Name) <= 16 THEN`<br>`  RETURN Name`<br>`  ENDIF`<br><br>`  // Otherwise it has to be trimmed`<br><br>`  WHILE LENGTH(Name) > 13`<br>`  REPEAT`<br>`  Name ← LEFT(Name, LENGTH(Name) - 1) // strip last char`<br>`  UNTIL RIGHT(Name, 1) = Space // back to SPACE`<br>`  ENDWHILE`<br><br>`  Name ← LEFT(Name, LENGTH(Name) - 1) // remove the space`<br>`  Name ← Name & Dots`<br>`  RETURN Name`<br><br>`ENDFUNCTION`<br><br>Mark as follows:<br>1. Function heading, ending, parameter and return type<br>2. If length of original string <= 16 then return original string<br>3. Any Conditional loop...<br>4. until string is short enough<br>5. Inner conditional loop / second condition to identify word(s) to remove from the end of string // Check to identify word(s) to remove from the end of string<br>6. Attempt to strip characters back to space<br>7. Correct removal of word/words from end of string **and** remove final space<br>8. Concatenate `Dots` and return result<br><br>**Max 7 marks** | **7** |
| 6(b)(i) | A (very) large **file** is created // redundant zeroes are stored in the file | **1** |
| 6(b)(ii) | One mark for:<br>• Values are delimited by a special character / a separator character<br>• First character indicates **sample** length<br><br>**Max 1 mark** | **1** |
| 6(b)(iii) | The **algorithm** to **store / extract / separate** the individual values is more complex / takes longer to execute / run / process<br><br>NE Algorithm is more complicated | **1** |

<table>
<tr><th>Question</th><th>Answer</th><th>Marks</th></tr>
<tr><td>7(a)</td><td>Examples include:<br><br>Module: <code>IdentifyMember()</code><br>Use: Identifies a club member who has expressed an interest in a given class<br><br>Module: <code>GetMemberPhoneNumber()</code><br>Use: Gets the mobile phone number of a member<br><br>Module: <code>CreateMessage()</code><br>Use: Generates a text message to a member<br><br>Module: <code>SendMessage()</code><br>Use: Sends a text message to a member in the waiting list<br><br>One mark for name and use<br><br>Note: max 3 marks</td><td>3</td></tr>
<tr><td>7(b)(i)</td><td><table><tr><th>Input</th><th>Output</th><th>Next state</th></tr><tr><td></td><td></td><td>S1</td></tr><tr><td>Input-A</td><td>none</td><td>S3</td></tr><tr><td>Input-A</td><td>Output-W</td><td>S3</td></tr><tr><td>Input-B</td><td>none</td><td>S2</td></tr><tr><td>Input-B</td><td>none</td><td>S5</td></tr><tr><td>Input-A</td><td>none</td><td>S2</td></tr><tr><td>Input-A</td><td>Output-X</td><td>S4</td></tr></table><br>One mark per row 3 to 7</td><td>5</td></tr>
<tr><td>7(b)(ii)</td><td>Input-B, Input-A</td><td>1</td></tr>
</table>

| Question | Answer | Marks |
|---|---|---|
| 8(a) | One mark for reason, one for benefit<br><br>Reason: (Program is) easier to design / implement / test / debug / modify<br><br>Benefit: Easier to check that **each stage** works as expected | **2** |

| Question | Answer | Marks |
|---|---|---|
| 8(b) | Example algorithm based on finding position of first non-space character and then using substring function:<br><br>`FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING`<br>`  DECLARE NewLine : STRING`<br>`  DECLARE EndOfLeading : BOOLEAN`<br>`  DECLARE Count, NumSpaces : INTEGER`<br>`  DECLARE NextChar : CHAR`<br>`  CONSTANT Space = " "`<br><br>`  NumSpaces ← 0`<br>`  EndOfLeading ← FALSE`<br><br>`  FOR Count ← 1 TO LENGTH(Line)`<br>`  NextChar ← MID(Line, Count, 1)`<br>`  IF NextChar <> Space AND EndOfLeading = FALSE THEN`<br>`  NumSpaces ← Count - 1 // the number to trim`<br>`  EndOfLeading = TRUE`<br>`  ENDIF`<br>`  NEXT Count`<br><br>`  NewLine ← RIGHT(Line, LENGTH(Line) - NumSpaces)`<br><br>`  RETURN NewLine`<br>`ENDFUNCTION`<br><br>Mark as follows:<br><br>1 Loop to length of parameter `// Loop until first non-space character in Line`<br>2 Extract a character **in a loop**<br>3 Identify <u>first</u> non-space character **in a loop**<br>4 Attempt at removing leading spaces in `Line`<br>5 Leading spaces removed from `Line` / / Create new string without leading space<br>6 Return a string following a reasonable attempt at removing leading spaces in `Line` | **6** |

| Question | Answer | Marks |
|---|---|---|
| 8(c) | Example:<br><br>`PROCEDURE Stage_2(F1, F2 : STRING)`<br><br>`DECLARE Line : STRING`<br>`DECLARE Count : INTEGER`<br><br>`Count ← 0`<br><br>`OPEN F1 FOR READ`<br>`OPEN F2 FOR APPEND`<br><br>`WHILE NOT EOF(F1)`<br>`READFILE F1, Line`<br>`Line ← DeleteSpaces(Line)`<br>`Line ← DeleteComment(Line)`<br>`IF Line <> "" THEN`<br>`WRITEFILE F2, Line // skip blank lines`<br>`ELSE`<br>`Count ← Count + 1`<br>`ENDIF`<br>`ENDWHILE`<br><br>`CLOSEFILE F1`<br>`CLOSEFILE F2`<br><br>`OUTPUT Count, " blank lines were removed"`<br><br>`ENDPROCEDURE`<br><br>Mark as follows:<br><br>1 Procedure heading, parameters, ending<br>2 Open both files in correct modes **and** subsequently close<br>3 Loop to `EOF(F1)`<br>4 Read a line from `F1` **in a loop**<br>5 Assign return values from `DeleteComment()` and `DeleteSpaces()` **in a loop**<br>6 Check return value following both MP5 function calls is not an empty string and if so write to `F2` **in a loop**<br>7 Count the blank lines **in a loop**<br>8 Output number of blank lines removed following a reasonable attempt **after the loop** | **8** |

© Cambridge University Press & Assessment 2024 